

Mock Design: An E-Commerce Checkout

Phase 4 · Interview-Ready · Estimated time: 2 hours

1. Concept From First Principles: Applying the Framework

Checkout is a genuinely different flavor of problem from the real-time and feed systems of the last three chapters: instead of optimizing for delivery speed or read throughput, the central tension is **correctness under concurrency** — specifically, never selling the same unit of inventory twice, while still keeping checkout fast and available during high-traffic sale events (a Big Billion Days or Flipkart Republic Day Sale-style spike is the natural framing for this exact problem).

Step 1 — Clarify requirements

- Functional: add items to cart; check out (reserve inventory, collect payment, create order); handle order cancellation/refund.
- Non-functional: never oversell inventory (correctness is non-negotiable); remain available and responsive during extreme flash-sale traffic spikes; payment must be handled safely, without double-charging.

Step 2 — Estimate scale

Assume steady-state 5,000 checkouts/second, but explicitly call out that flash sales (a new phone drop, a festival sale) can spike this 20–50x for short bursts — 100,000–250,000 checkouts/second briefly. This spike ratio, stated explicitly, is the number that justifies nearly every design decision in this chapter, the same way the read:write ratio drove Chapter 8's design.

Step 3 — API

POST /cart/items, POST /checkout {cartId, paymentMethodId} -> {orderId, status}. Note that checkout is deliberately a single, atomic-feeling API call from the client's perspective, even though (as Step 4 shows) it triggers a multi-step internal workflow.

Step 4 — High-level design

Client → Load Balancer → Checkout Service, which orchestrates three internal steps: (1) reserve inventory, (2) charge payment, (3) create the order record. Given the multi-service nature of this flow and this program's established preference for availability over blocking 2PC at real throughput (Phase 3, Ch 4), this orchestration uses a **Saga**: each step either succeeds and moves forward, or fails and triggers compensating actions for any already-completed steps (e.g., release the reserved inventory if payment fails).

Step 5 — Deep dive: inventory consistency and payment integration

Never overselling inventory is the hardest, most probed part of this design. A naive "read stock count, check if > 0, decrement" is a classic race condition: two concurrent checkouts can both read stock = 1 before either writes, and both proceed to sell the same last unit. The fix is an **atomic decrement** at the database level — a single conditional update statement (UPDATE inventory SET stock = stock - 1

WHERE product_id = X AND stock > 0) that only succeeds if stock is genuinely available, with the affected-row-count telling the application whether the reservation succeeded — eliminating the read-then-write race window entirely. At very high contention on a single hot product (a flash-sale item), even this atomic update can become a serialization bottleneck on one database row; a further optimization is to pre-shard a hot product's stock count across multiple counter rows/keys (similar in spirit to Chapter 4's rate limiter counter-sharding) and sum across shards, trading a small amount of allocation precision for much higher write concurrency.

Payment integration reuses Phase 2 Chapter 9's idempotency pattern directly: every checkout attempt carries a client-generated idempotency key, so a client retry after a timeout (the payment may have actually succeeded server-side even though the client never received the response) doesn't result in a double charge — the payment gateway recognizes the repeated key and returns the original result instead of processing a second charge. The **reserve-then-confirm** pattern is standard: inventory is reserved (soft-held, not yet permanently decremented) for a short window (e.g., 10 minutes, directly reusing Chapter 14's seat-locking pattern) while payment processes; if payment succeeds, the reservation is confirmed permanently; if it fails or times out, the reservation is released back to available stock.

Step 6 — Trade-offs and wrap-up

Volunteer: during an extreme flash-sale spike, even atomic per-row updates on the single hottest product can become the bottleneck — a queue-based admission system (accept checkout attempts into a queue, process them in order rather than all-at-once against the database) is a real mitigation worth naming, trading a short added wait for guaranteed correctness under extreme load rather than the database falling over entirely.

2. Real-World Example / Case Study

Flipkart's Big Billion Days and Meesho's festive sale events are widely known, real, extreme instances of exactly the flash-sale spike this chapter estimates against — publicly discussed engineering retrospectives from Indian e-commerce platforms consistently describe inventory-consistency-under-load and payment-idempotency as the two hardest problems in scaling checkout for these events, directly matching this chapter's Step 5 deep dive.

3. Diagram

```

Checkout Saga (orchestrated):
[Checkout Service]
|
|--(1)--> Reserve inventory (atomic conditional UPDATE)
|           success? continue : fail fast, no charge
|
|--(2)--> Charge payment (idempotency key)
|           success? continue : compensate -> release stock
|
|--(3)--> Create order record
|           success? done : compensate -> refund + release stock

Hot product under flash-sale contention:
[Stock: 10,000 units] split into N sharded counters
[Shard 1: 2500] [Shard 2: 2500] [Shard 3: 2500] [Shard 4: 2500]
(checkout hits a shard at random, atomic decrement per shard)

```

4. Trade-offs Table

Decision	Alternative	Why this choice
Atomic conditional UPDATE for stock decrement	Read-then-write in application code	Eliminates the race window that causes overselling
Saga for multi-step checkout	Two-Phase Commit	Avoids 2PC's blocking behavior at real checkout throughput
Sharded stock counters for hot products	Single stock row per product	Avoids single-row serialization bottleneck under extreme contention
Reserve-then-confirm inventory	Decrement permanently at reservation time	Allows clean release back to stock if payment fails or times out

5. Common Interview Questions

Q1: "How do you guarantee you never sell the same unit of inventory twice?"

Model approach: Name the atomic conditional update explicitly and explain precisely why a naive read-then-write has a race condition it doesn't have — this is the single most important sentence in this chapter's deep dive.

Q2: "What happens if payment fails after inventory has already been reserved?"

Model approach: Describe the Saga's compensating action — release the reservation back to available stock — and note this is exactly why reservation is a temporary soft-hold rather than an immediate permanent decrement.

Q3: "How would you handle a single product suddenly getting 100,000 simultaneous checkout attempts?"

Model approach: Name sharded stock counters to relieve single-row contention, and a queue-based admission system as a further mitigation if contention still exceeds what sharding alone can absorb.

6. Practice Exercises

- 1 Run the full six-step framework for "design an e-commerce checkout system," out loud, with a timer.
- 2 Write the exact SQL (or pseudocode) for the atomic conditional stock-decrement statement, and explain what the affected-row-count tells the application.
- 3 Design the compensating-action logic for a Saga where payment succeeds but order-record creation fails afterward.

7. Curated YouTube Videos

Language	Video & Channel	Why it helps
English	"E-Commerce Platform (Amazon, eBay): System Design Interview (Stripe & Amazon Offers)"	Covers checkout, inventory, and payment integration touchpoints directly.
English	"Amazon Interview Question System Design: Inventory Management (with FAANG Senior Engineer)"	Deep dive specifically on inventory consistency under concurrency.
Hindi	"System Design Interview: Architecture of Amazon, Flipkart like e-commerce system" — Gaurav Sen / InterviewReady	India-based instructor covering an Amazon/Flipkart-style e-commerce architecture.
Hindi	"System Design Complete Course 2026" (Hindi + English, 10+ hrs)	Bilingual course covering checkout and inventory as part of the broader curriculum.

8. Key Takeaways

- Checkout's central tension is correctness under concurrency (never oversell), not raw read/write throughput.
- Atomic conditional updates eliminate the read-then-write race condition that causes overselling.
- A Saga orchestrates the multi-step checkout flow with compensating actions, avoiding 2PC's blocking cost.
- Payment idempotency keys prevent double-charging on client retries after ambiguous network failures.
- Reserve-then-confirm inventory allows clean release of stock if any later step in the Saga fails.
- Sharded stock counters relieve single-row contention on extremely hot products during flash sales.
- Flash-sale spike ratios (20-50x steady state) are the number that justifies nearly every design decision here, echoing how the read:write ratio drove Chapter 8.